

Tutorial Sheet -2

The **AI Interview Assistant** is a **MERN stack-based** web application designed to provide **AI-powered mock interviews**. It evaluates user responses, provides real-time feedback, and tracks interview performance. To ensure a structured and efficient development process, we must choose an appropriate **Software Development Life Cycle (SDLC) model**.

For this project, we choose the **Iterative Incremental Model** as the preferred SDLC approach.

Justification for Choosing the Iterative Incremental Model

The **Iterative Incremental Model** is well-suited for this project due to the following reasons:

i. Flexibility in Requirement Changes

The AI Interview Assistant involves **integration of AI and speech-to-text technologies**, where requirements may evolve based on feedback. The **incremental approach** allows us to **start with a basic prototype** and add features in stages.

ii. Early Functionality & Feedback

Initial versions can be released with core interview simulations, and additional features like **real-time scoring, analytics, and live interviews** can be added incrementally. Users can provide feedback after each iteration, helping us **refine AI feedback models**.

iii. Risk Reduction & AI Model Improvements

AI-based features like **NLP answer evaluation** require **continuous improvement**. Using an iterative approach, we can **train and test AI models** in stages. **Reducing project failure risk** by testing small increments rather than waiting for a full release.

iv. Parallel Development

Frontend and backend components can be developed and tested in parallel, ensuring faster development.

v. Faster Time-to-Market

The **MVP (Minimum Viable Product)** can be launched early with basic AI interview simulations, while advanced features (analytics, live interviews, payment integration) are developed incrementally.

Detailed Process & Parameters Considered

Stages in the Iterative Incremental Model

Phase 1: Planning & Requirement Analysis

- Define core functionalities such as:
 - AI-based mock interview simulation.
 - Speech-to-text processing for voice-based answers.

- User authentication and role-based access.
- Prioritize features based on complexity and importance.

Phase 2: Design

- Define system architecture:
 - MERN stack (MongoDB, Express.js, React.js, Node.js).
 - AI integrations (OpenAI API, Google Speech-to-Text, TensorFlow.js).
 - State management (Redux/Context API).
- Database schema design for storing interviews, responses, and feedback.

Phase 3: Implementation (Incremental Development)

The project will be **developed in iterations**, each focusing on specific features:

Iteration	Features Developed
Iteration 1	User authentication (JWT-based login/signup)
Iteration 2	AI-generated interview questions based on role selection
Iteration 3	Speech-to-text & NLP-based response evaluation
Iteration 4	Real-time feedback and scoring mechanism
Iteration 5	Interview history and analytics
Iteration 6	Admin panel for managing interviews and analytics
Iteration 7	Live mock interviews and chatbot integration
Iteration 8	Payment system (Stripe, PayPal) for premium features

Each iteration will undergo:

- **Coding & Unit Testing**
- **Integration Testing**
- **User Feedback Collection**
- **Bug Fixes & Enhancements**

Phase 4: Testing & Quality Assurance

- Unit Testing: Ensure individual modules work correctly.
- Integration Testing: Ensure AI processing integrates with backend APIs.
- Performance Testing: Optimize system for scalability.
- Security Testing: Encrypt user data, secure payments, and prevent unauthorized access.

Phase 5: Deployment & User Feedback

- Deploy the MVP version on Vercel (frontend) & Railway/Render (backend).
- Gather real user feedback to enhance AI-generated responses.

Phase 6: Continuous Monitoring & Improvement

- Track **user engagement & performance analytics**.
- **Optimize AI models** using real interview responses.
- **Enhance features based on feedback** (e.g., support for more job roles).

In Conclusion, The **Iterative Incremental Model** is the best choice for developing the **AI Interview Assistant** because it:

- **Allows early deployment** with basic features.
- **Handles evolving AI requirements** through multiple iterations.
- **Minimizes risk** by testing features incrementally.
- **Speeds up development** by enabling parallel work on different modules.

By following this approach, we ensure a **scalable, flexible, and user-friendly** AI-powered interview platform.